

A Database Management System for Hostel & Student Services

Design and implementation of a centralized platform that handles student records, room allocations, hostel fees, complaints, leave applications, and notices from a single relational database.

PRESENTED BY

Sruthika

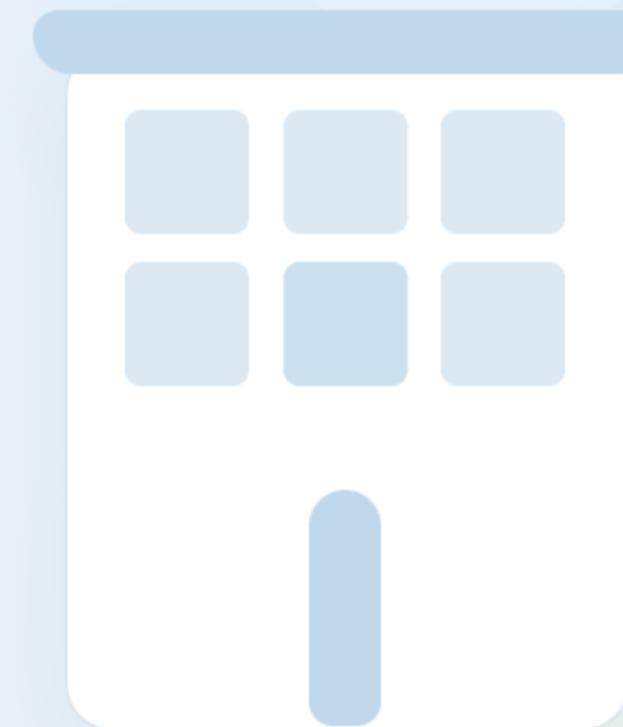
25WU0102312

Thrishith

25WU0102291

Sahasraa

25WU0101117



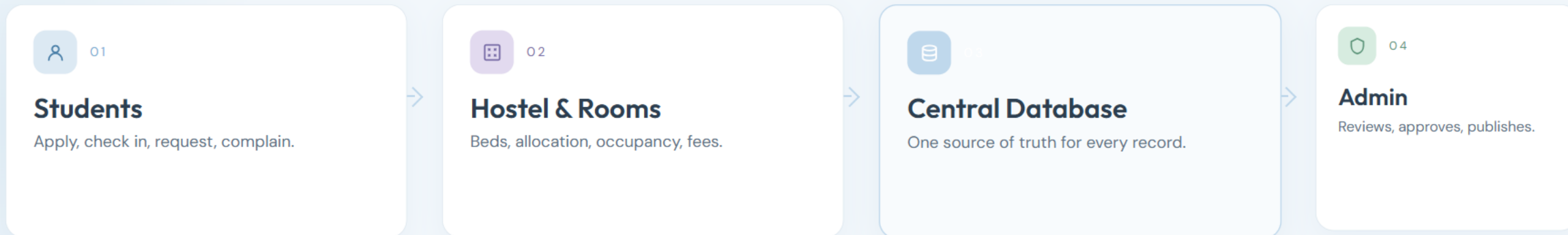
Introduction

A centralized database management system that brings the everyday operations of student accommodation into one place — from the first room assignment to the last outpass of the semester.

WHAT IT COVERS

- Student accommodation & room allocation
- Complaint & maintenance tracking
- Leave & outpass requests
- Notices & announcements

One flow, four touchpoints — from the resident to administration.



Problem Statement

Manual hostel operations are slow, error-prone, and scattered across registers, spreadsheets, and chat messages. Here's what breaks first.

• BEFORE · MANUAL SYSTEM

01 Records scatter across files

Student data lives in registers, sheets and chats — no single source of truth.

02 Beds are hard to track

Room and bed availability updates lag, leading to double-bookings.

03 Complaints fall through cracks

Verbal and paper complaints cannot be tracked, prioritised or audited.

04 Fees lost in registers

Who paid, who owes, who is overdue — all buried in ledgers.

05 Outpass records disappear

Leave approvals happen informally — no audit trail when it matters.

06 Admins lack a single view

Decisions are made from partial information, often too late.

• AFTER · CENTRALIZED DBMS

One database behind every decision.

Records, rooms, fees, complaints and notices — all readable, auditable and consistent from a single relational store.

Records

Allocations

Fees

Complaints

Leave

Notices

Objectives

Six concrete outcomes the system is designed to deliver — tied directly to the pains outlined in the previous chapter.



01

Centralize student records

One authoritative profile per resident — viewable, editable, exportable.



02

Manage rooms and beds

Track capacity, allocation and live occupancy block by block.



03

Track fees & dues

Semester payments and outstanding dues, always one query away.



04

Resolve complaints

Submit, track, update status — every maintenance request visible.



05

Handle leave requests

Outpass applications with approval workflow and a clear audit trail.



06

Publish notices

Hostel-wide announcements delivered and timestamped for everyone.

Proposed System

Three user roles, one shared platform. Each role sees only what it needs — but operates on the same underlying record.



A · ADMINISTRATOR

Admin

- Manage students, hostels and rooms
- Manage fees, complaints and notices
- Approve leave requests
- View operational reports



B · OPERATIONAL

Warden

- Monitor hostel occupancy and rooms
- Handle and update complaints
- Manage room allocations
- Review and act on leave requests



C · END USER

Student

- View room and hostel details
- Check fees and submit complaints
- Apply for leave or outpass
- Read notices, manage own profile

System Architecture

A thin client, a Flask-mediated backend and a relational store — a three-tier layout that's easy to demo, easy to host.



TIER 01

Users

Student · Warden · Admin



TIER 02

Frontend

HTML5 · CSS3 · JavaScript



TIER 03

Backend

Python · Flask



TIER 04

Database

Technology choices

• Zero-setup fallback included

The application is built to run on MySQL in production. For local demos without a database server, it falls back to SQLite — the schema and queries stay the same.

FRONTEND

HTML5

STYLING

CSS3

CLIENT LOGIC

JavaScript

LANGUAGE

Python

FRAMEWORK

Flask

AUTH

Sessions

PRIMARY DATABASE

SUPPORTED

MySQL 8.0+

Production-grade relational store — the target deployment.

DEMO FALLBACK

LOCAL ONLY

SQLite

Zero-setup alternative when no DB server is available.



Schema designed in **MySQL Workbench** · password hashing on every login.

Database Design

Nine relational tables, linked by primary and foreign keys. Each table owns one part of the system; together they describe a single hostel world.

NINE TABLES

9 tables



Users PK

Login credentials, hashed passwords and assigned role.



Hostel PK

Block-level information — name, location, type, capacity.



Room PK · FK

Room number, floor and capacity — links back to a hostel.



Student PK · FK

Personal details — name, roll, course, year, contact, parent info.



Room_Allocation PK · FK x2

Joins student to room with intake year and bed reference.



Fee PK · FK

Per-semester amount, paid date and balance for each student.



Complaint PK · FK

Subject, description, category, status and timestamps.



Leave_Request PK · FK

Outpass dates, reason, decision and the warden who approved.

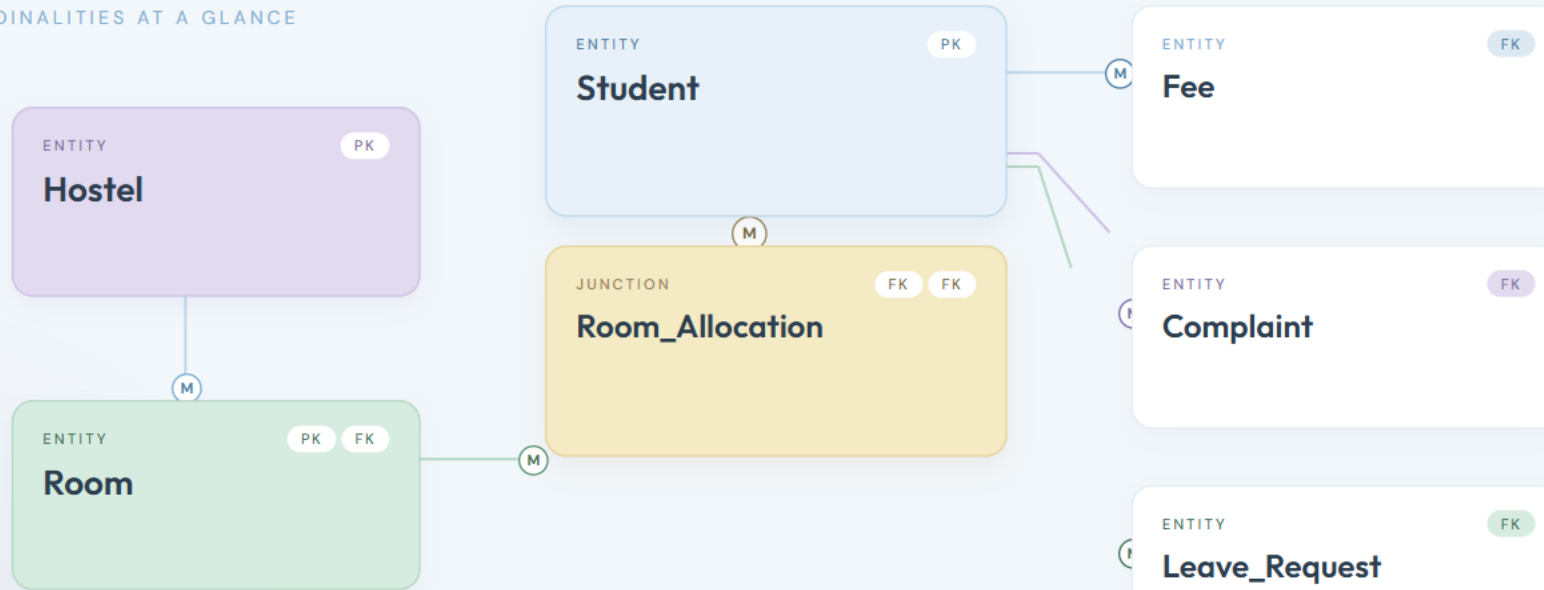


Notice PK

Title, body, audience scope and the publish timestamp.

Entity Relationships & DBMS Concepts

CARDINALITIES AT A GLANCE



ⓘ Each pair reads as **one** entity owning many — cardinalities repeat across every relationship.

CONCEPTS IN USE

PK Primary Keys

FK Foreign Keys

1 : M One-to-Many Relationships

RI Referential Integrity

CRUD CRUD Operations

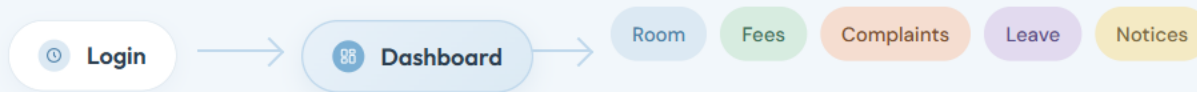
REL Relational Data Organisation

DATA Data Consistency

Student Portal

System implementation · what every resident sees and does.

FROM LOGIN TO ACTION



Student Dashboard

A personalised home with quick access to every feature.



Room & Hostel Details

See block, room number and roommate information instantly.



Fee Details

View semester fee, payment date and outstanding balance.



Submit & Track Complaints

Raise issues and follow status — open, in-progress, resolved.



Leave / Outpass

Apply with reason and dates, see approvals come back.



Hostel Notices

Read announcements as they're published, with timestamps.



Profile Management

Update personal info, contact details and emergency contacts.



Status Tracking

See live status of complaints and leave applications.

Warden / Admin Portal

The operations dashboard — every record, one screen away.

OPERATIONAL PULSE



Total Residents

Active students in residence.



Bed Occupancy

Live allocation status.



Open Complaints

Awaiting response or fix.



Pending Outpasses

Decision required.



Fee Collected

Across all students.



Outstanding Dues

Past-due balance.

FEATURES AVAILABLE



Student Records

Search, edit and export.



Rooms & Hostels

Blocks, rooms, capacity.



Room Allocations

Assign and reassign beds.



Complaints

Update status, close tickets.



Leave Approvals

Approve or reject.



Fee Management

Mark paid, view dues.



Notice Board

Compose and publish.



DBMS Reports

Operational summaries.



DATABASE HUB

Direct query & inspection

Read tables, review schema, run sanity checks — all from the admin view.

CRUD Operations

How the system actually talks to its database — four verbs, used across every screen.



INSERT

Create

Add new records to the system.

Add student records

Add rooms

Submit complaints

Create leave requests

Publish notices



UPDATE

Update

Change what is already on file.

Update student info

Update allocations

Update complaint status

Approve or reject leave

Update fee status



SELECT

Read

Retrieve and inspect existing data.

View student records

Check availability

View fees

View complaints

View leave requests



DELETE

Delete

Remove records when no longer needed.

Remove obsolete records

Withdraw leave

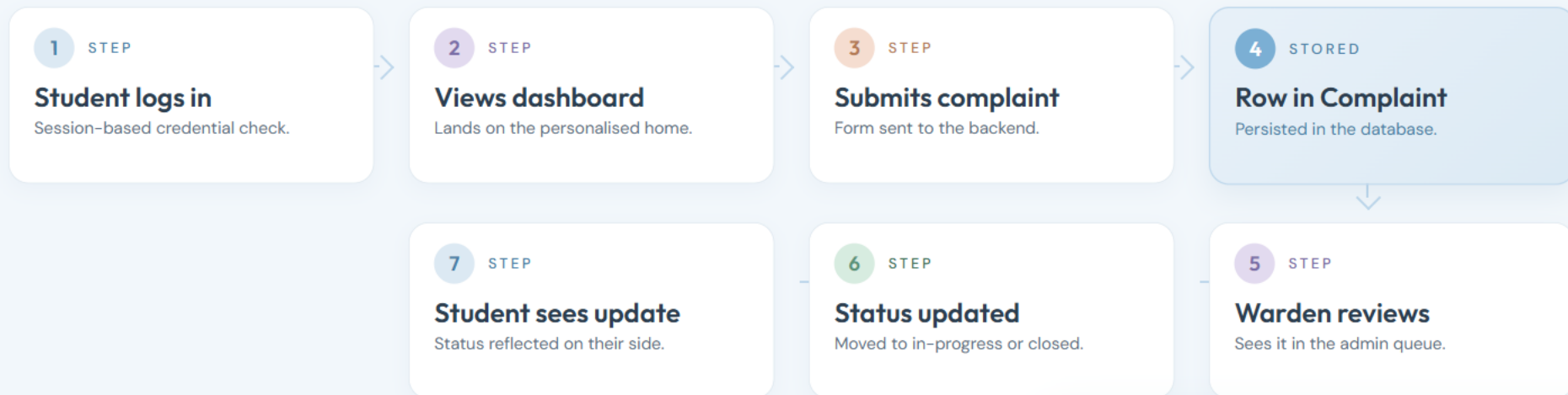
Archive notices

Used sparingly

System Workflow & Live Demo

One full complaint lifecycle, end to end — then what we'll show live.

COMPLAINT LIFECYCLE



WHAT WE'LL SHOW LIVE

- 01 Student login
- 02 Room details
- 03 Fee status
- 04 Complaint submission
- 05 Warden dashboard
- 06 Status update
- 07 Leave request
- 08 Database records

END OF DECK

Thank You.

Questions & Answers

Listening for the committee's review.